

Single Chip Video Controller

Bob Haas
20887 SW Willapa Way
Tualatin OR 97062

Several semiconductor manufacturers have recently produced video display controllers contained on a single integrated circuit. While none of these is the "video terminal-on-a-chip" that some of the publicity would have you believe, these new devices perform many of the functions required in a video display, thereby reducing the number of integrated circuit packages required. In addition, they are all programmable to some degree, which allows adding new features to an existing design at low incremental cost, or changing display formats if required by changing needs. These characteristics make these devices particularly interesting to a computer experimenter. A user might start with a 16 line, 64 character, upper case only display, and as requirements (and budget) increase, convert to

a "professional" 24 line, 80 character, upper and lower case format just by adding more memory and a new character generator.

In this article, I will present a survey of the characteristics of four video display controllers, namely, the Intel 8275, the Motorola MC6845, the NS (National Semiconductor) DP8350, and the SMC (Standard Microsystems Corporation) 5027. In addition, I will present a detailed description of the Motorola part and a design for a display using that device.

Device Characteristics

Table 1 summarizes the important characteristics of each device. All of the devices are programmable. The Intel, Motorola, and SMC parts are programmed by the microprocessor system to which they are attached. This means that when the system is powered up, a program must be executed to initialize the display controller, before a proper display will appear on the display screen.

The display formats of the Intel and Motorola devices are, within limits, completely variable. For the Motorola part, any line width from 1 to 256 characters can be chosen. (Of course, these limits are unreasonable values; the actual limits are determined by the display timing constraints, an example of which will be given in the detailed discussion of the Motorola unit, later in this article.) The SMC 5027 is manufactured with a fixed "menu" of line widths,

Text continued on page 56

About the Author:

Bob Haas leads a double life: by day he designs 6800 systems (for Tektronix's 4051 group); and by night he works on 6502 systems (on his own KIM-1 system). He uses the MC6845 display system described in this article with his KIM-1.

Acknowledgment

The author wishes to thank Mr Hank Trieckel of Motorola, Inc for his assistance in the preparation of this article.

	Intel 8275	Motorola MC6845	National Semiconductor DP8350	Standard Microsystems Corp 5027
Format Lines by Characters	fully programmable to 64 by 80	fully programmable to 128 by 256	mask programmable to 64 by 110	programmed options to 64 by 132
Microprocessor Compatibility	8080 family (direct memory access only)	all	all	all
Simplest System Interface	direct memory access via Intel 8257	shared memory	shared memory	shared memory
Display Memory Size (maximum)	64 K	16 K	4 K	4 K
Addressing	linear	linear	linear	row/column
Scrolling	line, character, page	line, character, page	line, character, page	line only
Cursor	blink or steady. reverse video or underline	blink or steady. reverse video or underline	reverse video or underline, no blink	reverse video or underline, no blink
Interlace	none	video, or video and sync, or none	none	none or interlaced sync and video
Light Pen	yes	yes	no	no
Graphics Capability	limited	full	full	full
Process	MOS	MOS	III	MOS
Power	+5 V	+5 V	+5 V	+5, +12 V
Other Features (see text)	on board line buffers; visual attributes		high-speed timing on board	self-loading for stand- alone use

Table 1: Summary of characteristics of four video display controllers.

Text continued from page 52:

such as 20, 32, 40, 64, 72, 80, 96, and 132 characters, from which the initialization program can choose.

The NS DP8350 is mask programmed. Its characteristics are set by internal read only memories, the contents of which are determined when the device is manufactured. Changing the display format with this part means unplugging the current unit and plugging in a differently manufactured unit. A number of stock formats, among them 24 lines by 80 characters, will be available, but if your particular needs are not met by a stock part, you must contract with NS to program a custom part, agree to purchase a certain quantity, and perhaps pay a one time mask charge.

The SMC part has the ability to load its format parameters at power-up from an external read only memory, so that it does not have to be part of a microprocessor based system at all. It can therefore be used in a so-called "dumb" terminal.

All of the devices may theoretically be interfaced to any microprocessor, but practicalities limit the choices. The Intel part, being part of the 8080 family, is designed to interface to 8080 based systems via DMA (direct memory access) through the Intel 8257 DMA controller. This makes it difficult to interface the Intel part to non8080 systems. In fact, since many people's S-100 bus 8080 systems will not support direct mem-

ory access, the Intel part would be difficult to interface even to these systems.

The Motorola MC6845, a member of the Motorola 6800 family, is easily interfaced to 6800 and 6502 systems, and can be interfaced to 8080 and Z-80 systems. The NS and SMC parts have system interfaces similar to the Motorola part. The simplest method of interfacing the latter three parts is by means of shared memory, wherein the display memory appears to the processor to be ordinary programmable memory.

Memory Usage

The maximum size of the display refresh memory for each part is limited by the number of refresh memory address lines coming out of the package. The Motorola part has 14 address lines, and the NS and SMC parts each have 12. The Intel part has access to the entire system memory through an attached direct memory access controller, and the system memory may be as large as 64 K (65,536) bytes. The Intel, Motorola, and NS parts access linear (sequential) refresh memory addresses, so there is a simple relationship between the refresh memory address of a given character and its position on the display screen.

The SMC 5027, however, outputs addresses in a row and column format which (without the addition of hardware to do address translation) causes inefficient use of

display memory when the display dimensions are not integer powers of 2. For example, building a 24 line by 80 character display with the SMC part would require a refresh memory that is 24 by 128, or 3,072 bytes total. Of these, 1,152 bytes would not be displayed. Also, a program to manage the display would have to perform row and column calculations to locate a given screen position. Of course, additional hardware could be added to "linearize" the addresses, but this defeats the purpose of using one of these devices, namely, the reduction of hardware.

Scrolling and Cursing

All of the devices provide scrolling, that is, the ability to move data around on the display screen without actually moving the data in the refresh memory. The SMC device provides line scrolling only. An example of scrolling using the Motorola part is given in a later section of this article.

All the parts provide for the generation of a *cursor* (ie: some way for a human operator to determine the position at which the next character entered from a keyboard will be placed on the display). The Intel and Motorola devices allow a steady or blinking cursor consisting of an underscore or a reverse video (black on white) block. The NS and SMC parts allow for underscore or reverse video, but do not provide blinking.

Video Signal Characteristics

A standard North American television picture is composed of two frames of 262½ scan lines each, with scan lines from alternate frames interlaced vertically the width of one line, so that the resultant picture has 525 scan lines. Many computer video displays use only about 262 scan lines, and are not interlaced. This limits the maximum number of character rows on a display to about 25.

The Motorola part allows the use of interlacing to produce an aesthetically more pleasing display by doubling character dots vertically. This fills in what might otherwise be spaces on a high-resolution display. This is called *interlaced sync*, in Motorola's terminology. The Motorola and SMC parts also provide for *interlaced sync and video*, in which all 525 lines can be used for character formation, allowing as many as perhaps 50 character rows per display. The use of interlace does cause a flicker effect on ordinary white phosphor (P4) monitors, but computer experimenters with long persistence, green phosphor (P39) monitors may want to consider using interlace.

Other Features

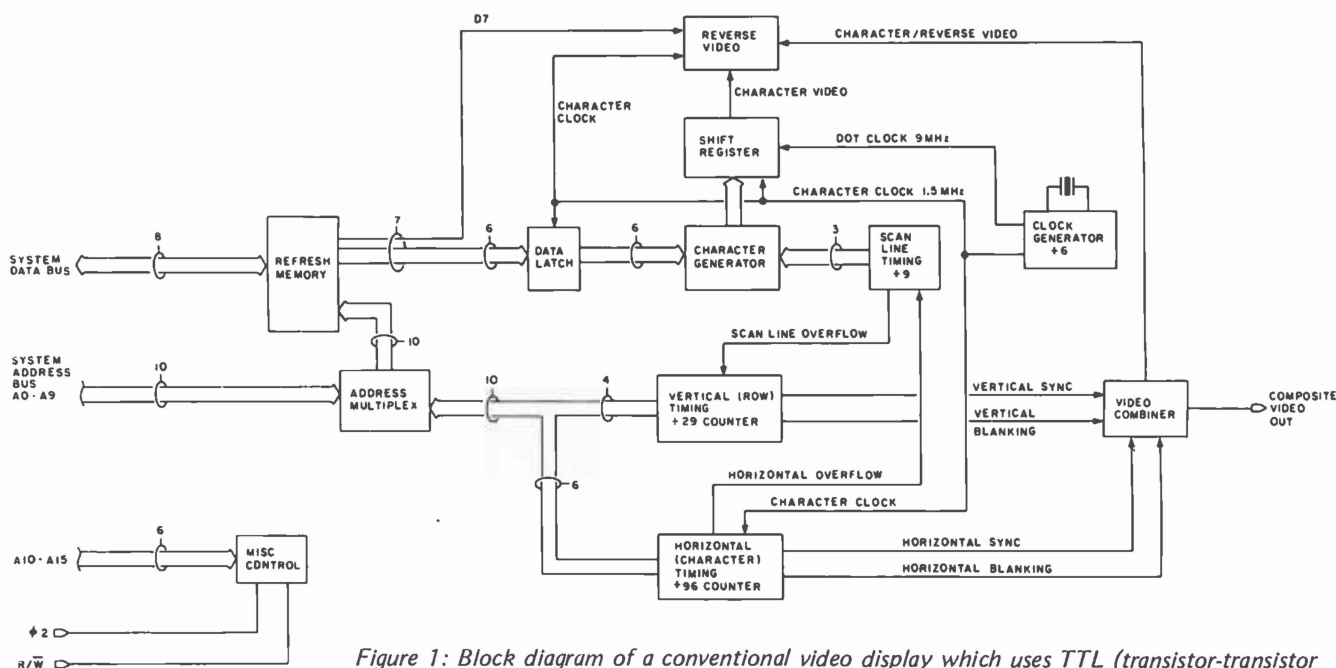
The Intel and Motorola circuits provide support for a *light pen*, that is, a light-sensitive "wand" used by the display operator to point to areas on the display screen to signify something to the attached system. This requires additional hardware to implement.

The data displayed on the screen by the display controller need not be dot patterns from a character generator read only memory. They might be dots forming part of a graphic image. Except for the Intel part, the display controllers support whatever graphics-generation hardware the system designer cares to attach to them. The Intel part is limited in this area because of its line buffers, which orient it toward character generation only (more on this later).

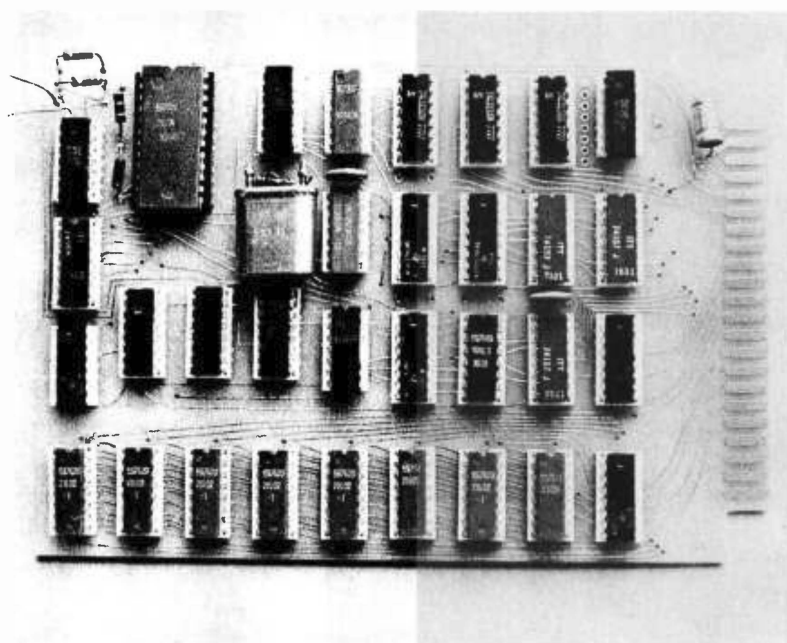
The Intel, Motorola, and SMC display controllers are manufactured by the MOS (metal oxide semiconductor) process, and do not include the so-called high-speed timing function of a display on the device. The National part, however, uses an I²L (integrated injection logic), with none of the speed limitations of MOS, so it does include the high-speed timing functions. This inclusion helps to reduce external parts count. (A discussion of just what these timing functions are follows in a later section of this article.)

As mentioned earlier, the Intel display controller must interface to a system through a DMA controller such as the Intel 8257. The Intel display controller incorporates two 80 character line buffers. While it is displaying a row of characters from one line buffer, it fills the second line buffer from the memory by "stealing" some memory cycles. It then uses the second line buffer for display and fills the first line buffer from the third row of characters, and so on. The timing for a 24 line by 80 character display is such that up to 25 percent of a system's memory cycles may be taken by the display controller action. The Intel part's line buffers store 7 bit characters, so the graphics achievable with this part are limited to what can be displayed with a 128 character set, augmented by character-set switching (using additional hardware).

A distinguishing feature of the Intel part is support for *visual attributes*. With only minimal external hardware, blocks of characters can be made to blink, be highlighted (higher than normal brightness), be reversed (black on white), be underlined, or have any combination of these four qualities. In addition, two more attribute signals are provided that could provide color selection on a color display.



A Conventional Display Design



display circuit must perform, and how those functions can be carried out with conventional TTL (transistor-transistor logic). Figure 1 shows the block diagram of such a display. A total of 32 packages (including the memory and character generator circuits) is required. The design is optimized for a 16 line, 64 character format. It uses the shared memory type of interface to the system. This interface scheme is simpler to implement, and faster for system updates, than the cursor control interface used in TV Typewriter designs, but it does suffer from a "snow" effect when the system updates the display.

The horizontal timing circuit is driven by the character clock. This circuit counts to 95, then resets itself to 0; therefore there are a total of 96 character intervals in each horizontal scan. The character clock rate of 1.5

MHz (9 MHz divided by 6) yields a character time of $0.666 \mu\text{s}$. 96 character intervals per horizontal scan yield a scan time of $63.94 \mu\text{s}$, or a frequency of 15,640 Hz, close to the television standard. Of the 96 character intervals, 64 are displayed, and 32 are blanked.

At the end of each horizontal scan, another counter, the scan line counter, is incremented. It counts to 8, then resets itself to 0. Its output is connected to the character generator, to cause the character generator to output the correct line of dots for each scan line. Scan lines 0 and 8 are blank, because the character generator puts out no dots for these lines. There are, therefore, two blank scan lines between rows of characters. When the scan line counter resets to 0, the vertical (row) counter is incremented. The vertical counter counts to 28, then resets to 0. The first 16 counts are used to display character rows, and the remaining 13 are blanked. The 29 rows of nine scan lines each yield a total of 261 scan lines per frame, a vertical scan time of 16.69 ms ($63.94 \mu\text{s}$ times 261), and a vertical frequency of 59.9 Hz, close enough to 60 Hz to minimize any shimmy problems in the display caused by power supply ripple or magnetic fields.

The addresses supplied to the refresh memory are produced by the horizontal (character) timing and the vertical (row) timing. The vertical address is incremented only when the scan line counter resets, so

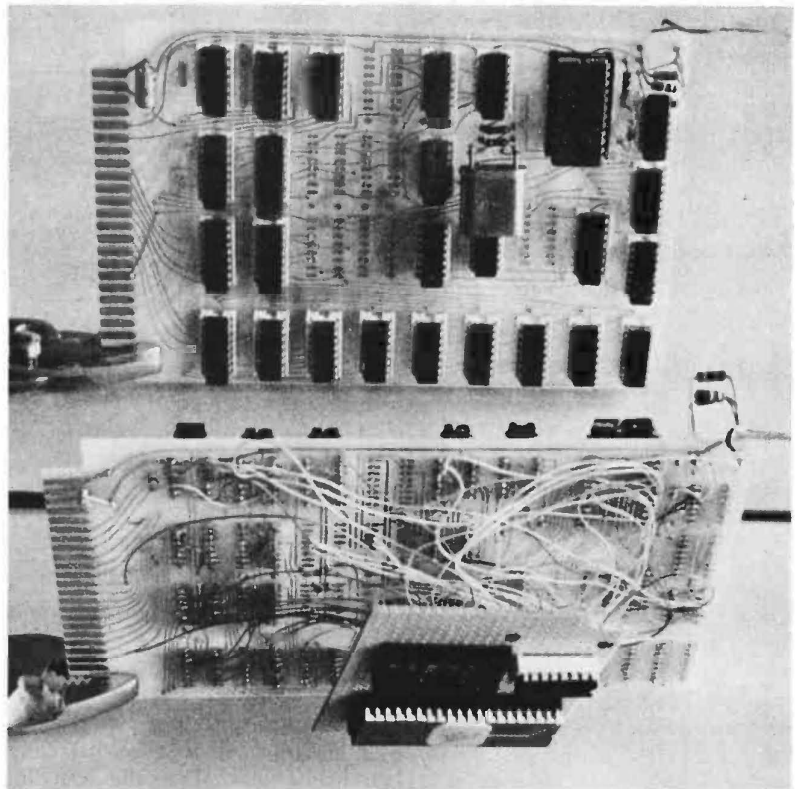
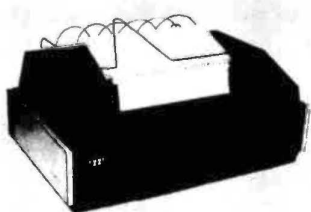


Photo 2: Printed circuit board from photo 1 after modification. Observe the wiring changes and "kluge board" attached to the back side of the board. The small kluge board contains the Motorola MC6845 video display processor, which has been grafted onto the original circuit. A mirror was used to show front and back sides simultaneously.

TELETYPES®

IMMEDIATE DELIVERY

MODEL 40 300 LPM PRINTERS



- Mechanism or complete assembly
- 80-column friction feed
- 80-column tractor feed
- 132-column tractor feed

INTERFACES

- EIA-RS232
- Simplified EIA-like interface
- Standard serial interface
- Parallel device interface

MODEL 43 TERMINALS



- 4310 RO (Receive Only)
- 4320 KSR (Keyboard Send-Receive)
- 4340 BSR (Buffered Send-Receive)

FEDERAL Communications Corporation

11126 Shady Trail, Dallas, Texas 75229, (214) 620-0644,
TELEX 732211 TWX 910-860-5529

INTERFACES

- TTL Serial
- EIA RS232 or DC20 to 60ma
- 103-type built-in modem

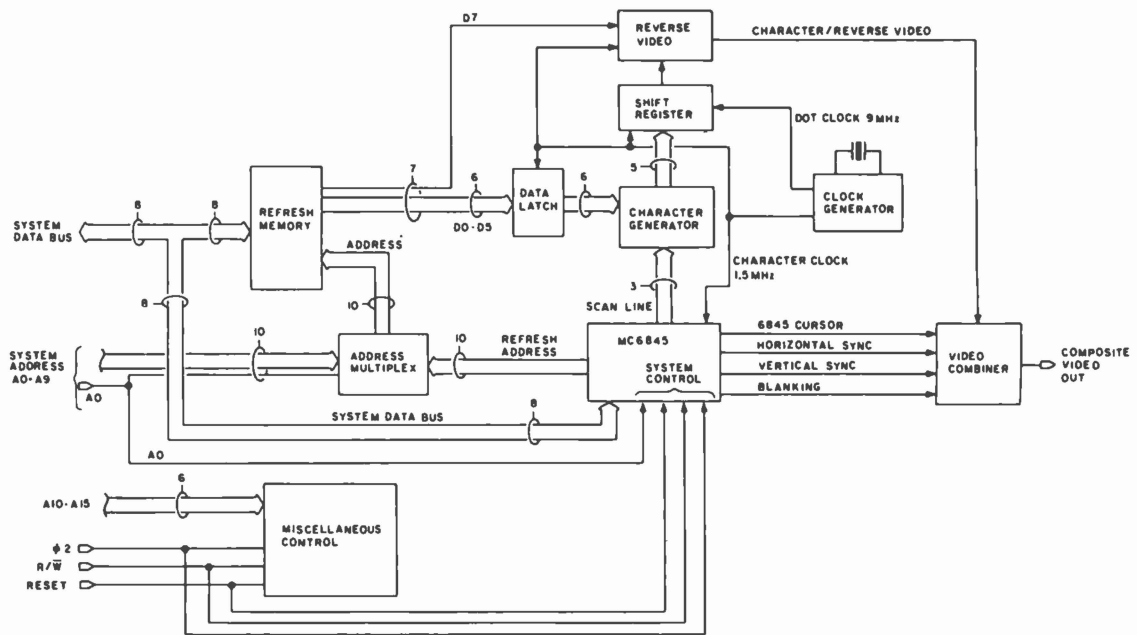


Figure 2: Block diagram of a video display utilizing the Motorola MC6845 video display controller.

that a given row of 64 characters of refresh memory is scanned nine times, in order to "paint" all seven scan lines of the characters, plus two blank lines. When the scan line counter resets, the vertical counter is incremented so that the next row of 64 characters may be scanned. In order to center the display, the vertical sync pulse is produced at about the 22nd character row, and the horizontal sync pulse at about the 80th character interval.

The circuit includes the capability of reversing (ie: converting to black on white) any character with bit 7 (the most significant bit) on. This can be used to highlight blocks of text, or generate a cursor.

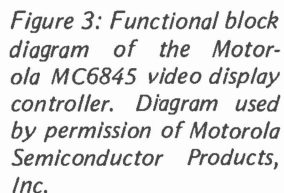
Normally the refresh memory is connected to the vertical and horizontal timing circuits through a *multiplexer*, which can be thought of as a 10 pole, 2 position switch. When the processor wants to update the display, control circuitry switches the multiplexer so that the address the processor wants to update is supplied to the refresh memory instead of the address the timing circuits would be supplying. The processor reads or writes the location. The memory output is probably not correct for the display at that moment, so a segment of a different character is substituted for the correct one, producing the snow effect if extensive updates are being performed. The snow can be eliminated by allowing the processor to access the refresh memory only when the display is blanked, but I did not include circuitry for this in my design.

Using the Motorola MC6845 Display Controller

Figure 2 shows the block diagram of the display redrawn using the Motorola MC6845. The change is not striking. The 6845 has replaced only three blocks, namely, scan line timing, vertical (row) timing, and horizontal (character) timing. The circuit using the 6845 has five fewer packages. The 6845 occupies about the same amount of board space, consumes about the same amount of power, and costs more than the TTL packages it replaces. What have we gained by the replacement? For the person who is perfectly happy with the 16 by 64 TTL design, nothing. However, the advantage of the 6845 lies in its programmability. The characteristics of the display of which it is a part are easily changeable. This means that the same circuit can provide formats other than 16 by 64, such as 25 by 40, 14 by 72, and 12 by 80. The 6845 provides hardware scrolling, a blinking cursor (in addition to the selectable reverse video carried over from the all TTL circuit), support for a light pen, and three interlace options.

The programmability of the 6845 is in one way a slight disadvantage. An initialization program must be run by the system before the display will start up. The TTL version starts displaying immediately upon power-up, although the display will show at first whatever random characters the refresh memory contains at power-up.

Motorola's diagram of the 6845's internal



In order for the system to access the

64 May 1979 © BYTE Publications Inc

Listing 1: Initialization routine for MC6845 as coded for the 6800 microprocessor.

0200	5F		CRTINI	CLR	B		INIT REG #
0201	CE	0213		LDX		#TABLE	INIT TABLE PTR
0204	F7	E800	CRTLTP	STA	B	CRTC	STORE REGISTER NUMBER
0207	A6	00		LDA	A	0,X	GET TABLE VALUE
0209	B7	E801		STA	A	CRTC+1	STORE IN REGISTER
020C	08			INX			NEXT TABLE VALUE
020D	5C			INC	B		NEXT REGISTER NUMBER
020E	C1	10		CMP	B	#16	LAST REGISTER?
0210	26	F2		BNE		CRTLTP	REPEAT IF NOT LAST
0212	39			RTS			RETURN TO CALLER
0213			TABLE	FCB		\$5E,\$40,\$4D,\$08	
0217				FCB		\$1C,\$02,\$10,\$16	
021B				FCB		\$00,\$08,\$40,\$08	
021F				FCB		\$00,\$00,\$00,\$00	

Listing 2: Initialization routine coded for the 6502 microprocessor.

0200	A2	00	CRTINI	LDX	#0	INIT REG AND TAB PTR
0202	8E	00	E8 CRTLP	STX	CRTC	STORE REGISTER NUMBER
0205	BD	11	02	LDA	TABLE,X	GET TABLE VALUE
0208	8D	01	E8	STA	CRTC+1	PUT VALUE IN REGISTER
020B	E8			INX		NEXT REGISTER
020C	E0	10		CPX	#16	LAST?
020E	D0	F2		BNE	CRTLTP	REPEAT IF NOT
0210	60			RTS		RETURN TO CALLER
0211			TABLE	.BYTE	\$5E,\$40,\$4D,\$08	
0215				.BYTE	\$1C,\$02,\$10,\$16	
0219				.BYTE	\$00,\$08,\$40,\$08	
021D				.BYTE	\$00,\$00,\$00,\$00	

Photo 3: Display generated by MC6845 controlled circuit. The 12 line by 80 character format is shown. Parameters illustrated are values placed in 6845 registers by program executing on the main system processor. A 10.275 MHz crystal is used to correct for severe over-scan in the author's monitor.

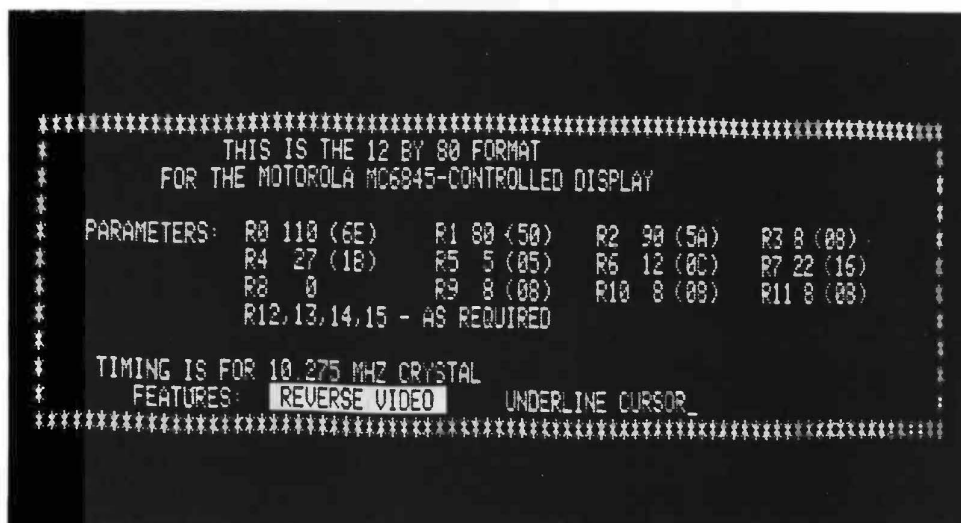
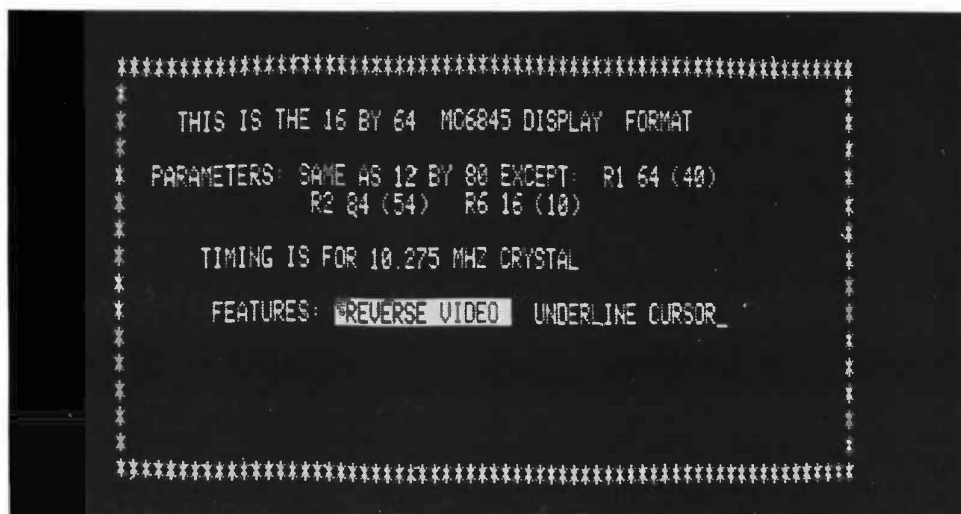


Photo 4: MC6845 circuit generated display using 16 line by 64 character format. Parameters shown and referenced are correct for 10.275 MHz crystal, which was left in place after demonstrating 12 by 80 format. See table 2 for parameters appropriate for 9.0 MHz crystal.



these latter addresses X and X+1; with the 6845's RS (register select) line connected to system address line 0 (the least significant bit), these will be consecutive addresses.

The first address, X, is the 6845's "pointer" register, which determines which register is accessed through address X+1. To write to a particular register, store the register number at X, and the desired value at X+1. A routine to initialize the 6845 coded for the 6800 is shown as listing 1, and a version for the 6502 as listing 2. The 6502 version is slightly shorter, because the 6502's X register can be used both as a table pointer and as the 6845 register number.

Table 2 summarizes the function of each register and the values to be programmed into each register for three formats: 16 lines by 64 characters, 25 by 40, and 12 by 80. The detailed function of each register and the calculation of the values for the 16 by 64 format are as shown in the following seventeen examples.

R0 Horizontal total. This register is programmed with one less than the total number of character intervals in a horizontal scan. A value of 94 provides for a total of 95 character intervals. This produces slightly better overall timing than the value of 96 character intervals used in the TTL circuit. The horizontal scan time is $95 \times 0.666 \mu s = 63.27 \mu s$, for a frequency of 15,800 Hz.

R1 Horizontal columns displayed. A value of 64, equal to the number of characters displayed, is used.

R2 Horizontal sync position. A value of 77 specifies that the horizontal sync pulse is to start at the 77th character position. This value centers the display on my particular monitor, but may be varied as needed for other monitors.

R3 Horizontal sync width. This is specified in number of character intervals. A value of 8 yields a sync pulse width of $5.33 \mu s$, close to the television standard.

R4 Vertical total. This register is programmed with one less than the total number of character rows. A value of 28 specifies 29 character rows.

R5 Vertical total adjust. This register allows adding additional scan lines to the vertical display time to trim the vertical scan frequency, if required to bring it close to the power line frequency (to minimize display "shimmy"). A value of 2 is used here.

R6 Vertical rows displayed. This register is programmed with the number of character rows to be displayed, 16.

R7 Vertical sync position. A value of 22 specifies that the vertical sync pulse is to be produced at the 22nd character row, which centers the display on my monitor. Other monitors may require a slightly different

Register	Function	Value for 16 by 64	25 by 40	12 by 80
R0	horizontal total	94 (5E)	—	110 (6E)
R1	horizontal cols. displayed	64 (40)	40 (28)	80 (50)
R2	horizontal sync position	77 (4D)	66 (42)	90 (5A)
R3	horizontal sync width	8 (08)	—	—
R4	vertical total	28 (1C)	—	27 (1B)
R5	vertical total adjustment	2 (02)	—	5 (05)
R6	vertical rows displayed	16 (10)	25 (19)	12 (0C)
R7	vertical sync position	22 (16)	27 (1B)	22 (16)
R8	interlace mode	0 (0)	—	—
R9	maximum scan line	8 (08)	—	—
R10	cursor start	64 (40)	—	—
R11	cursor end	8 (08)	—	—
R12	refresh start address (high order)	0*	—	—
R13	refresh start address (low order)	0*	—	—
R14	cursor address (high order)	0*	—	—
R15	cursor address (low order)	0*	—	—
R16	light pen (high order)	**		
R17	light pen (low order)	**		

Table 2: Appropriate values to be stored in internal registers of MC6845 for several display formats. The decimal representation is given first, and the hexadecimal representation follows in parentheses. The values marked by one asterisk () may be updated during display usage. The positions marked by two asterisks (**) are for a light pen; this design does not provide for a light pen. The values given for the 12 line by 80 character format should be used with a 10.275 MHz crystal. The values for the 16 by 64 and 25 by 40 formats are specified for a 9.0 MHz crystal.*

Value Stored in R10 (decimal)	Result
0	nonblinking reverse block
8	nonblinking underscore
32 or 40	no cursor
64	fast blinking reverse block
72	fast blinking underscore
96	slow blinking reverse block
102	slow blinking underscore

Table 3: Summary of cursor options for Motorola MC6845. To produce results shown in table, a value of 8 must also be stored in R11.

value. The vertical sync pulse width is not programmable, as is the horizontal sync pulse width. It is fixed at 16 scan line times.

R8 Interlace mode. This register is programmed with 0, specifying no interlace (equivalent to the TTL circuit). Two other interlace modes are available, as mentioned previously, but these require a long persistence phosphor (P39) monitor.

R9 Maximum scan lines. This register



Photo 5: Display demonstrating 25 line by 40 character format, again produced by the MC6845 circuit. As before, see table 2 for 9.0 MHz crystal parameters.

is programmed with the maximum scan line number that is to be presented to the character generator, and is 1 less than the number of scan lines per character row. A value of 8 causes the counter to run from 0 to 8, then back to 0. This produces a total of nine scan lines per character row. Using this number along with the others specified above, the resultant vertical timing is: 29 rows (R4) X nine lines per row (R9) = 261 scan lines. 261 scan lines + two lines extra (R5) = 263. Now, $263 \times 63.27 \mu\text{s}$ per scan line = 16.64 ms per vertical scan, or a frequency just under 60.1 Hz, again close to the power line frequency. (Other values could be used to adjust the vertical frequency to 50 Hz, the common power line frequency in other countries.)

R10 and R11 Cursor start and end. These registers specify the format of the cursor. The values of 64 for R10 and 8 for R11 generate a cursor which is a blinking reverse video block covering the entire character. For discussion of other cursor options see the section entitled "Cursor Generation."

The above registers are write only. Values may be stored in them, but not read back. These registers are generally written to only once (when the system is first powered up) to establish the characteristics of the display.

R12 and R13 Refresh start address. These

registers are the high order six bits (R12) and low order eight bits (R13) of a 14 bit refresh address counter. For a nonscrolled display, these are initialized to 0. For a scrolled display, these registers will be updated periodically; since they, too, are write only, copies of them must be maintained by the processor. More information on the use of these registers is given in the section on scrolling.

R14 and R15 Cursor location. These registers are the high order six bits (R14) and low order eight bits (R15) of the location at which the cursor is to be displayed. When the refresh address output by the 6845 equals the cursor address, a cursor output signal is activated, subject to the constraints placed on the cursor by values placed in R10 and R11. More information on cursor generation is in a later section of this article. R14 and R15 are in principle readable as well as writeable, but unless proper buffering is provided for the 6845, they cannot be read. The circuit presented in the next section does not have the proper buffering, so these registers are treated as if they were write only, and copies are maintained by the processor.

R16 and R17 Light pen. These read only registers capture the refresh memory address at the instant a pulse is received from an external light pen. The processor can thereby calculate where on the display screen the operator is pointing the light pen. I provide no circuitry to support this feature of the 6845.

Display Design Description

Figure 4 shows the schematic of the display using the Motorola MC6845. The MC6845 being a MOS device (limited in counting speed to about 3 MHz), the higher speed dot and character clock circuits are still TTL. These are the high-speed timing functions mentioned previously. They must be implemented with external TTL packages on the Intel and SMC parts as well. The National display controller includes the dot clock crystal oscillator and the character clock divider on the chip.

IC10 is a 9 MHz crystal oscillator. IC9 and IC7a divide this by 6 to produce the character clock. IC9 counts from 0 to 5; when it reaches 5, the output of IC7a goes low, conditioning IC9 to reset itself on the next clock pulse. Two variants of the character clock are used. The output of IC7a goes high when IC9 goes from 5 back to 0, and a rising edge clock pulse is needed for IC2, IC4, and IC5. A falling clock is needed for IC1 and the 6845. Furthermore, the clock supplied to IC1 must be high a minimum of 220 ns, and low a minimum of 160

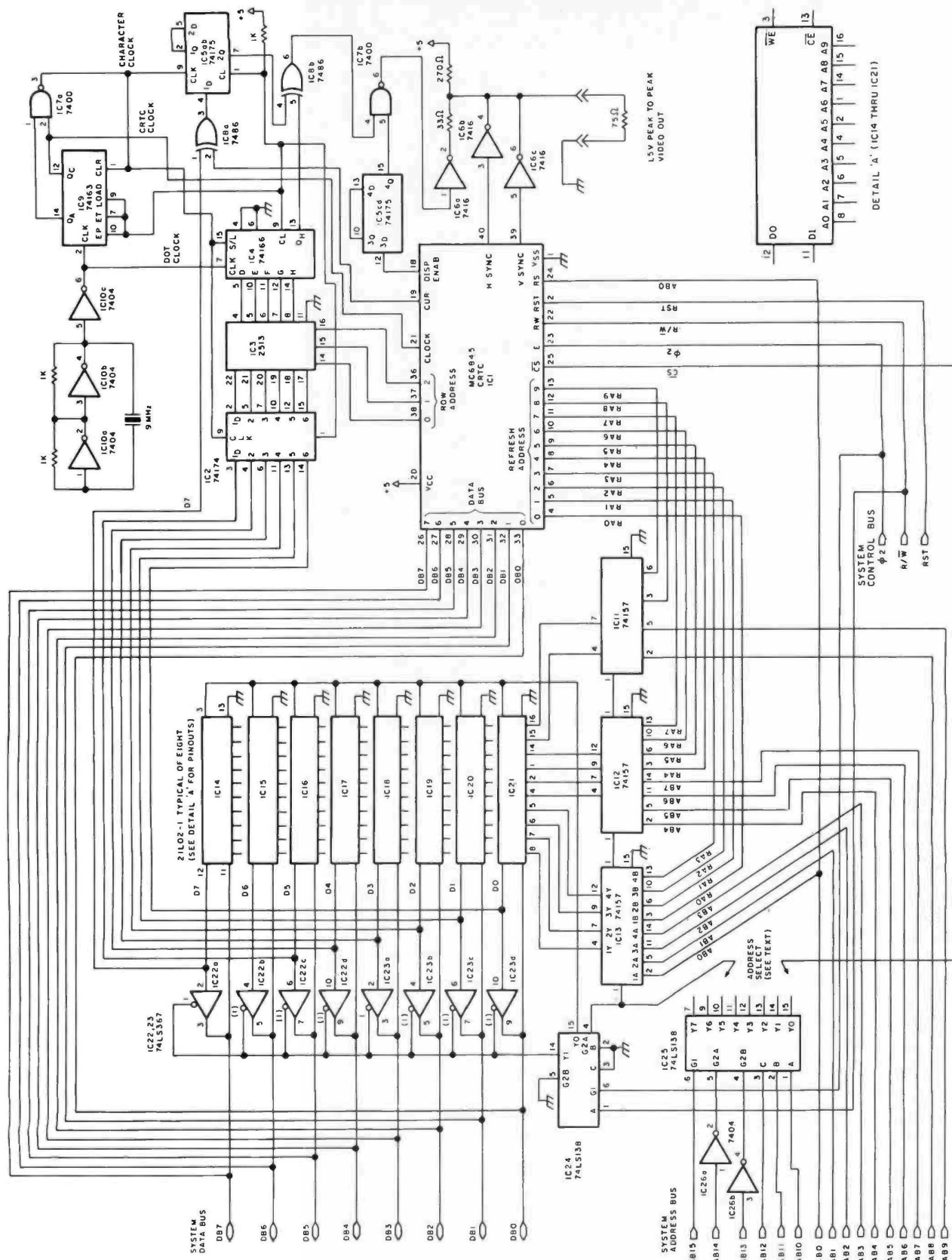


Figure 4: Schematic diagram of display circuit incorporating the MC6845 device. All integrated circuits except IC6 may be low power Schottky (LS) type.

Number	Type	+5 V	GND	-12 V	-5 V
IC1	MC6845	20	1		
IC2	74174	16	8		
IC3	2513	24	10	1	12
IC4	74166	16	8		
IC5	74175	16	8		
IC6	7416	14	7		
IC7	7400	14	7		
IC8	7486	14	7		
IC9	74163	16	8		
IC10	7404	14	7		
IC11	74157	16	8		
IC12	74157	16	8		
IC13	74157	16	8		
IC14	21L02-1	10	9		
IC15	21L02-1	10	9		
IC16	21L02-1	10	9		
IC17	21L02-1	10	9		
IC18	21L02-1	10	9		
IC19	21L02-1	10	9		
IC20	21L02-1	10	9		
IC21	21L02-1	10	9		
IC22	74LS367	14	7		
IC23	74LS367	14	7		
IC24	74LS138	16	8		
IC25	74LS138	16	8		
IC26	7404	14	7		

Table 4: A power pin table for the circuit in figure 4.

ns. The output of IC7a is of the wrong phase and is low for only 110 ns. Fortunately, the C output of IC9 is high for 220 ns (during counts 4 and 5), so it is used as the 6845 clock.

The 6845 presents the address of a character for refreshing the display to the memory. The memory then presents the character to the latch, IC2. The character in IC2 is then presented to the character generator, IC3. The dots for the specified scan line of the character are presented to the shift register, IC4, and shifted out at the dot clock rate to produce the video signal.

The 2 Character Pipeline

There is effectively a 2 character pipeline — one character being accessed from the refresh memory, and one character (actually one row of dots from a character) being accessed from the character generator. The two 6845 signals, *cursor* and *display enable*, must be delayed two character intervals by sections of IC5 to accommodate the pipeline effect. *Display enable* is low whenever the display is to be blanked. This includes the regions below, above, to the left, and to the

right of the active display area. *Cursor* is high when the current refresh address matches the value programmed into the cursor address register pair (R14 and R15). Bit 7 of the character, the 6845 *cursor* signal, and the video signal from the shift register are combined in such a way that bit 7 being on causes reversal of the video for one character interval (changing white-on-black characters to black-on-white, or vice versa), and the 6845 *cursor* signal being on causes another reversal. Assuming the cursor has been so programmed, it can be distinguished from ordinary reversal because it will blink.

One timing consideration must be borne in mind when the MC6845 is used. The counter used in the all TTL circuit has negligible delay (20 ns) compared to the display character time (666 ns). The MC6845, being a metal oxide semiconductor device, is considerably slower, with a delay of as much as 160 ns. This delay time must be subtracted from the character time when specifying the refresh memory access time. The refresh memory integrated circuits specified in the design are “-1” suffix types (500 ns maximum access time) so the timing is satisfactory.

On the schematic diagram (figure 4), IC26 (74LS138) and IC27 (7404) are connected such that IC27 is enabled for the uppermost 8 K bytes of processor memory address space (hexadecimal E000 thru FFFF). Other connections of IC26's enable inputs (pins 4, 5, and 6) to the address lines, with or without sections of IC27, as required, can allow enabling for any 8 K memory address segment. Selection of a particular 1 K byte segment for the refresh memory is accomplished by connecting the refresh memory select line to a particular output of IC26. The CS (chip select) line from IC1, the 6845, is connected to another of the outputs of IC26. This allocates an entire 1 K byte segment to the 6845, whereas it needs only two addresses. More integrated circuits could be added to refine the decoding for the 6845 and eliminate the wasted address space.

Cursor Generation

The MC6845 provides several options for the generation of a cursor. Registers R10 and R11 control the format of the cursor, and R14 and R15 control its position. The low order five bits of R10 (bits 0 thru 4) specify the scan line on which the cursor is to start, and R11 specifies the scan line on which the cursor is to end. If R10 bits 0 thru 4 are all equal to 0, and R11 is 8, the cursor will occupy lines 0 thru 8, or the entire character. Using the circuitry presented earlier, the cursor becomes a block

of reverse video. If the value 8 is stored in R10 bits 0 thru 4 (ie: bit 3 is on) and in R11, the cursor occupies only line 8. Hence it becomes an underscore. If values other than these are used, only a portion of the character is reversed. I have found that partial reversal makes characters difficult to read, so the only values I consider usable are (0, 8) and (8, 8).

Bits 5 and 6 of R10 control cursor blinking. If bit 5 is on and bit 6 is off, the cursor is not displayed at all. This can be used to blank the cursor to indicate the system is not accepting keyboard input. If bit 6 is on, the cursor will blink. If bit 5 is off, the blink rate is about four times per second. If bit 5 is on, the blink rate is about two times per second.

Scrolling

Scrolling is the management of a video display in the following way. New data is entered on the bottom line of the display. When the bottom line is full, the entire display is moved up one line. In the process, the top line, containing the oldest data, may be discarded, or if the display memory is larger than the portion displayed on the screen, the old display data may temporarily

be kept. In the latter case, the display could be scrolled down as well as up, and the display screen could act as a moveable "window."

The data movement necessary to implement scrolling could be done by a processor program. In fact, it must be done by the processor in the case of the all TTL display design, for there is no provision for hardware scrolling in that design. A program loop to perform scrolling on a 1,024 character screen might take from 15 to 20 milliseconds on a 6502 or 6800 processor. If the source of data to the screen was a serial communications line operating at 1200 bits per second (assume the system is emulating a terminal), the time between characters is only about 8 ms, not long enough to perform a scrolling operation. (An interrupt-driven program could be written to handle both scrolling and receiving of characters from the line, but this would be complex).

The 6845 does provide scrolling because its refresh start address is programmable, and may be updated whenever necessary. Up to this point, I've used the term *scrolling* to mean "line scrolling" where data is moved around as complete lines. In this case, the refresh start address of the 6845 would be updated in increments of 64 (for the 64 character line length).

However, scrolling can be done by individual characters. If the refresh start address is incremented by one, each character in each line moves one position left, and the first character of each line moves to the last position of the previous line. Also, if the display memory is at least twice as large as the display screen, scrolling could be done by page, in which case the refresh start address would be updated by 1,024 each time (again assuming the 16 by 64 format). Since the 6845 can address up to 16 K (16,384) bytes, the refresh memory could contain up to 16 pages of data, and scrolling could be done by line or page.

The design I have presented here has a refresh memory the same size as the display screen. It uses scrolling to enter new data on the bottom line of the display, and the top line is discarded when it is displaced. An example of how such scrolling operations might be done is shown in figure 5. Figure 5a shows how the display would be initialized. The 6845 is initialized with a refresh start address of 64 (decimal). The 6845's refresh address counter runs to 1,023 at the end of the 15th line, then continues with 1,024 and up to 1,087 at the end of the 16th and last line. Since only ten of the 6845's 14 refresh address lines are connected to the refresh memory, a *wraparound* occurs — the address 1,024 is equivalent (in

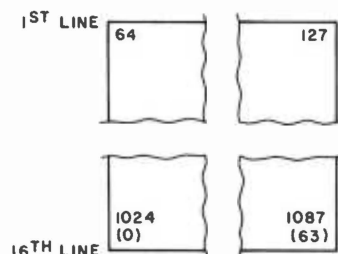


Figure 5a: Initialization for a scrolled display. Refresh addresses are shown. The differing value given in parentheses is that perceived by the refresh memory, due to wraparound.

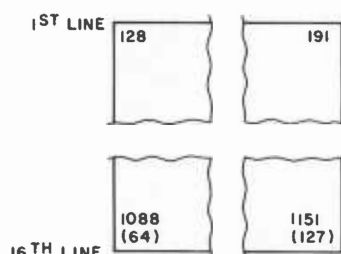


Figure 5b: Refresh addresses calculated after one scrolling operation.

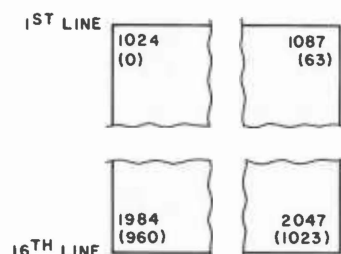


Figure 5c: Refresh addresses of last scrolling operation before the processor memory reference must be reset to 0.

the refresh memory's perception) to 0. Hence the last line of the display starts at a memory address of relative 0, from the processor's point of view. For example, if the display memory were located at processor hexadecimal addresses E000 thru E3FF, the last line of the display would start at E000.

The procedure to perform a scroll operation is as follows: increment the refresh start address by 64, update the cursor address, and prepare the processor to store new data at refresh memory locations 64 thru 127 (relative to the actual processor starting address; for the example given above, the addresses would be E040 thru E07F). Figure 5b shows the result of this scroll operation.

All addresses are incremented by 64 for each new line until the situation shown in figure 5c prevails. In order to perform another scroll operation, the processor memory address must be reset to relative 0 (E000 as above), but the 6845 refresh start address can continue to be incremented; it needn't be reset. It will eventually wrap around itself.

Note well that the cursor address register is a 14 bit register, as is the refresh start address register. All 14 bits of the cursor address must match a refresh address displayed on the screen for the cursor to be displayed. The range covered by the refresh address is determined by the refresh start address and the number of characters on the screen. If the cursor address is outside of this range, no cursor will be generated by the 6845.

Scrolling in the case of a 12 line by 80 character format (where neither of the dimensions is a power of 2) is more complicated. As shown in figure 6a, the 6845 is initialized with a refresh start address of 144, so that, at the beginning of the 12th line, the 6845 outputs the address 1024, which is equivalent to memory address 0. In figure 6b a single scroll operation has been performed. All values have been incremented by 80. So far, everything is just like the 16 by 64 case, except for the increment value. In figure 6c, the last "simple" scroll has occurred, and things get more complicated from this point. In figure 6d the result of another scroll operation is shown. Again, all values have been incremented by 80, but as can be seen, memory wraparound occurs within the display line. In the 16 by 64 case, wraparound always occurs between lines, and it is relatively easy for a processor program to deal with. In the 12 line by 80 character case, the processor program must be aware that wraparound can occur with a line, and it must act accordingly.

Device Availability

The SMC 5027 and the Intel 8275 (along with its associated 8257 controller) have been available for some time from computer hobbyist vendors. The 5027 was originally priced at about \$50, but may be available for less than that now in view of increasing competition. The Intel 8275 and 8257 pair are available for under \$100. The Motorola MC6845 is available from regular electronics distributors. It usually costs about \$30. The National DP8350 is the most recently announced of the four parts, so its price and availability may still fluctuate.

There are other video display controllers besides the four I have covered in this article. There will probably be even more announced by the time it is published — and prices can be expected to fall as competition heats up.■

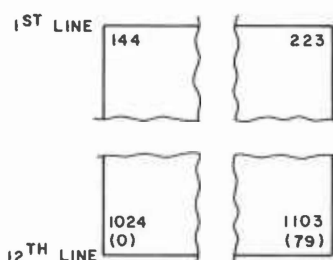


Figure 6a: Address initialization for a 12 line by 80 character display.

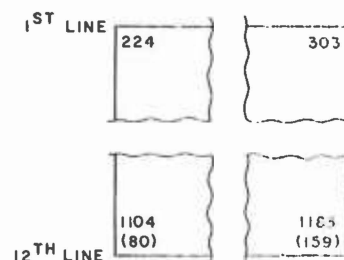


Figure 6b: Addresses in a 12 by 80 display following one scroll operation.

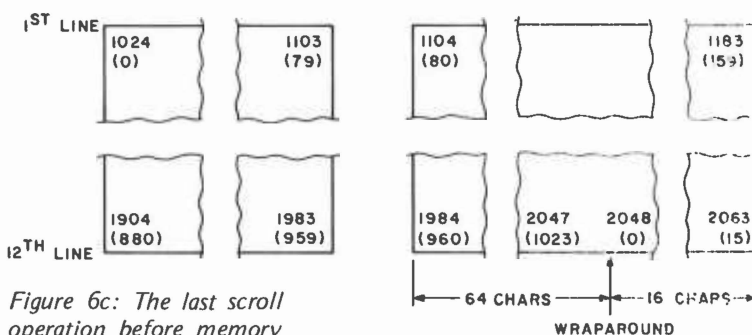


Figure 6c: The last scroll operation before memory wraparound occurs.

Figure 6d: Memory wrap-around. All values have been incremented by 80, but wraparound occurs within the display line. In the 12 line by 80 character format, the processor program must take special action at this wraparound event.